Operating System - Assignment 2

313553053 康峻瑋

1.Describe how you implemented the program in detail. (10%)

i. Parse program arguments

- 使用 `getopt` parse 傳入的 argv，`getopt` 會根據 `"n:t:s:p:"` 去區分arguments。
- 使用 `policy` 跟 `priority`這兩個array去存每個thread對應的policy跟priority

```
/* 1. Parse program arguments */
int opt;
while ((opt = getopt(argc, argv, "n:t:s:p:")) != -1) {
    switch(opt) {
        case 'n':
            num_thread = atoi(optarg);
            break;
        case 't':
            time_wait = atof(optarg);
            break;
        case 's':
            policies = optarg;
            break;
        case 'p':
            priorities = optarg;
            break;
    }
}

int policy[num_thread];
int priority[num_thread];

char *p;
p = strtok(policies, ",");
for (int i = 0; p != NULL; i++) {
    if (!strcmp(p, "NORMAL"))
        policy[i] = SCHED_OTHER;
    else if (!strcmp(p, "FIFO"))
        policy[i] = SCHED_FIFO;
    p = strtok(NULL, ",");
}

p = strtok(priorities, ",");
for (int i = 0; p != NULL; i++) {
    priority[i] = atoi(p);
    p = strtok(NULL, ",");
}
```

ii. Create <num_threads> worker threads

- 宣告後續需要的資料型態以及初始化`barrier`
- `pthread_barrier_init`是設定thread數量達到`num_thread+1`時才會同時執行
- `num_thread+1`是因為main thread也算一個thread

```
    /* 2. Create <num_threads> worker threads */
    pthread_barrier_init(&barrier, NULL, num_thread+1);
    pthread_attr_t pthread_attr[num_thread];
    struct sched_param param[num_thread];
    thread_info_t thread_info[num_thread];
```

iii. Set CPU affinity

- `CPU_ZERO()`為清空cpuset
- `CPU_SET(0, &cpuset)`將所有的thread放在`cpu 0`上執行
- `sched_setaffinity`設定sched的affinity

```
    /* 3. Set CPU affinity */
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(0, &cpuset);
    sched_setaffinity(getpid(), sizeof(cpuset), &cpuset);
```

iv. Set the attributes to each thread

- 創建`thread_info_t`用於存放每個thread相關屬性，在FOR LOOP中assign每個thread的屬性
- `pthread_attr_init`初始化所有的thread
- `pthread_setaffinity_np(3)`: 設定 CPU affinity
- `pthread_attr_setschedparam(3)`: 設定 scheduling parameters
- `pthread_attr_setschedpolicy(3)`: 設定 scheduling policy
- 透過`thread_func`去執行`print`跟`busy waiting`，`busy waiting`內容會在報告後續說明

```
typedef struct {
    pthread_t thread_id;
    int thread_num;
    float time_wait;
    int sched_policy;
    int sched_priority;
} thread_info_t;

void *thread_func(void *arg)
{
    /* 1. Wait until all threads are ready */
    thread_info_t *thread_info = (thread_info_t *) arg;
    pthread_barrier_wait(&barrier);
```

```
    /* 2. Do the task */
    for (int i = 0; i < 3; i++) {
        printf("Thread %d is starting\n", thread_info->thread_num);
        /* Busy for <time_wait> seconds */
        time_t start = time(NULL);
        while(time(NULL) - start < thread_info -> time_wait){
        }
    }
    /* 3. Exit the function  */
    return NULL;
}

/* 4. Set the attributes to each thread */
for (int i = 0; i < num_thread; i++) {
    pthread_attr_init(&pthread_attr[i]);
    thread_info[i].thread_num = i;
    thread_info[i].time_wait = time_wait;
    thread_info[i].sched_policy = policy[i];
    thread_info[i].sched_priority = priority[i];

    param[i].sched_priority = thread_info[i].sched_priority;
    pthread_attr_setinheritsched(&pthread_attr[i], PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&pthread_attr[i], thread_info[i].sched_policy);
    pthread_attr_setschedparam(&pthread_attr[i], &param[i]);
    pthread_create(&thread_info[i].thread_id, &pthread_attr[i], thread_func, (void
*)&thread_info[i]);
    pthread_setaffinity_np(thread_info[i].thread_id, sizeof(cpu_set_t), &cpuset);
}
```

v. Start and wait for all threads

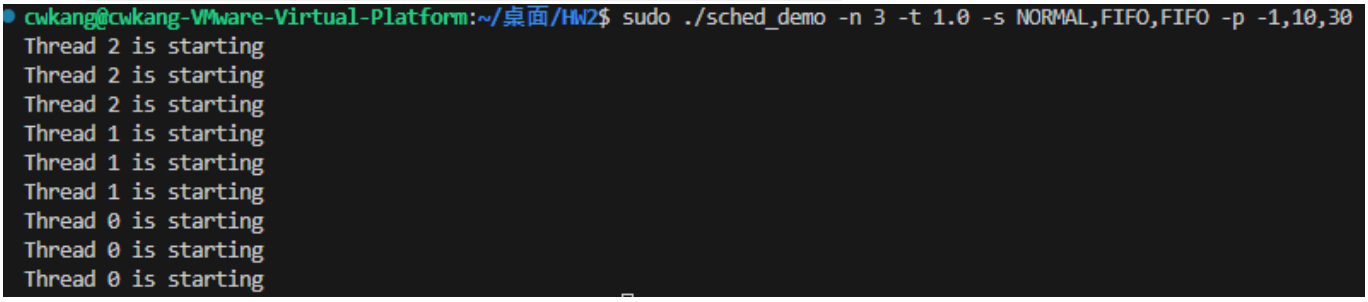
- `pthread_barrier_wait`開始執行所有的threads
- `pthread_join`等所有的thread執行完成
- `pthread_barrier_destroy`釋放barrier

```
    /* 5. Start all threads at once */
    pthread_barrier_wait(&barrier);

    /* 6. Wait for all threads to finish  */
    for (int i = 0; i < num_thread; i++) {
        pthread_join(thread_info[i].thread_id, NULL);
    }
    pthread_barrier_destroy(&barrier);
```

2.Describe the results of `./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30` and what causes that

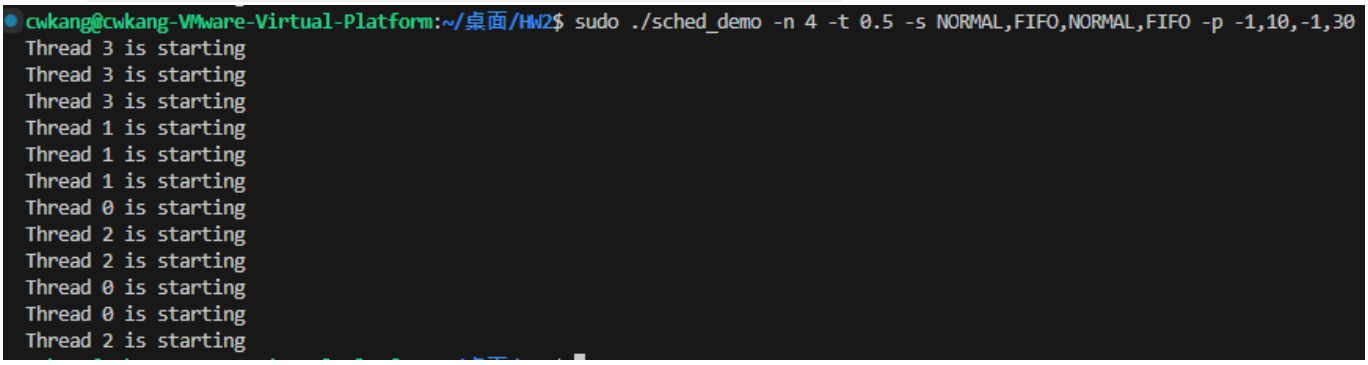
下圖為`sudo sysctl -w kernel.sched_rt_runtime_us=1000000`之後的結果，方便對照想法。



因為設定`sudo sysctl -w kernel.sched_rt_runtime_us=1000000`之後，FIFO thread一定會比NORMAL thread早執行，而FIFO thread會根據priority的高低決定執行順序，priority高的會優先執行，thread 2跟thread 1都是FIFO，而thread 2的priority比較高，所以thread 2先執行，然後換thread 1，最後是NORMAL thread的thread 0執行，與圖片中的輸出一致。

3.Describe the results of `./sched_demo -n 4 -t 0.5 -s NORMAL,FIFO,NORMAL,FIFO -p -1,10,-1,30`, and what causes that

下圖為`sudo sysctl -w kernel.sched_rt_runtime_us=1000000`之後的結果，方便對照想法。



因為設定`sudo sysctl -w kernel.sched_rt_runtime_us=1000000`之後，FIFO thread一定會比NORMAL thread早執行，而FIFO thread會根據priority的高低決定執行順序，priority高的會優先執行，thread 3跟thread 1都是FIFO，而thread 3的priority比較高，所以thread 3先執行，然後換thread 1，最後是NORMAL thread的thread 0跟thread 2需要執行，因為linux 是採取CFS的排班策略，所以thread 0跟thread 2會根據vruntime去交替執行，與圖片中的輸出一致。

4.Describe how did you implement n-second-busy-waiting?

我使用`time(NULL)`去取得現在時間的秒數`start`，使用while一直去計算現在時間`-start`去卡住thread，直到執行時間`>time_wait`才離開。

```
time_t start = time(NULL);
while(time(NULL) - start < thread_info -> time_wait){
}
```

5.What does the `kernel.sched_rt_runtime_us` effect? If this setting is changed, what will happen?

- `sched_rt_runtime_us` 指的是在 `sched_rt_period_us`，允許即時任務運行的時間，預設為 950,000 us（0.95 秒）。
- `sched_rt_period_us` 定義了該週期的總長度，預設為 1,000,000 us（1 秒）。

在預設設置下，即時任務每 1 秒最多可以運行 0.95 秒，留下 0.05 秒供普通任務使用，避免即時任務完全佔用 CPU。

- 設定為 950000：普通任務仍然能夠獲得部分 CPU 時間。
- 設定為 1000000：即時任務幾乎佔用全部CPU，普通任務需到等到即時任務全部完成之後才可以拿到 CPU。

例子如下，皆使用`sudo ./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30`

- kernel.sched_rt_runtime_us = 950000

```
cwkang@cwkang-VMware-Virtual-Platform:~/桌面/HW2$ sudo ./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30
Thread 2 is starting
Thread 2 is starting
Thread 0 is starting
Thread 2 is starting
Thread 1 is starting
Thread 1 is starting
Thread 1 is starting
Thread 0 is starting
Thread 0 is starting
```

- kernel.sched_rt_runtime_us = 1000000

```
cwkang@cwkang-VMware-Virtual-Platform:~/桌面/HW2$ sudo ./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30
Thread 2 is starting
Thread 2 is starting
Thread 2 is starting
Thread 1 is starting
Thread 1 is starting
Thread 1 is starting
Thread 0 is starting
Thread 0 is starting
Thread 0 is starting
```

可以從圖中發現當`kernel.sched_rt_runtime_us = 950000`時，雖然thread 0是NORMAL，但是當為FIFO的thread 2使用CPU時，CPU會有短暫的時間會讓給thread 0執行。

而`kernel.sched_rt_runtime_us = 1000000`時，NORMAL thread就一定要等到所有的FIFO thread執行完才可以拿到CPU去執行。

在`sched_test.sh`中也可以看到對於`sched_rt_runtime_us`的設定，為了方便觀察及比較，可以將`sched_rt_runtime_us`先設定為1000000，等shell腳本執行完之後再將設定改為預設的950000。